

SPL: Orchestrating Workflows with Declarative Deterministic–Probabilistic Composition

Anonymous authors

Paper under double-blind review

1 Abstract

We present SPL (Structured Prompt Language), a declarative language for orchestrating LLM-powered workflows that composes two fundamentally distinct computation modes — **deterministic** and **probabilistic** — within a single specification. Existing frameworks treat these modes separately: imperative orchestration systems (LangGraph, AutoGen, CrewAI) are built exclusively around probabilistic LLM calls, while symbolic tools (SymPy, SageMath, Lean) operate independently of the orchestration layer. SPL closes this gap through a declarative composition model. It provides **GENERATE** and **EVALUATE** for probabilistic computation (LLM invocation, semantic branching, quality-gated iteration) and **SOLVE** and **ASSERT** for deterministic computation (kernel-routed symbolic execution, formal verification). Both modes share the same syntax, the same variable bindings, and the same runtime adapter layer — the programmer declares *what* each step computes; the runtime routes it to the appropriate engine.

The language is governed by the DODA principle (Design Once, Deploy Anywhere): a `.spl` specification executes unchanged across local GPU nodes (Ollama), cloud APIs (OpenRouter, Anthropic), and a decentralized inference grid (Momagrid), with provider and verifier selection deferred to invocation time. We validate SPL through two lenses. First, a 65+ recipe cookbook spanning 12 workflow categories demonstrates expressiveness across both modes. Second, a controlled 1200-run experiment ($r=3$) — 10 models $\times$ 20 mathematical problems $\times$ 2 execution arms (solver vs. LLM-only) $\times$ 3 repeats across 6 difficulty tiers — provides quantitative evidence that the deterministic arm achieves 82–93% machine-verified correctness for the most capable models (sonnet-4-6: 85%, gemma4:e2b: 93%) while exposing a backend difficulty gradient: SymPy problems (T0–T2) reach 78% solver pass rate, Sage problems (T3–T5) reach 54%, reflecting the harder decomposition demands of transcendental and symbolic-algebra operations. The dominant failure mode is `solver_error` (kernel rejecting malformed expressions), not format non-compliance. The reference implementation is open-sourced under Apache 2.0.

Keywords: declarative language, LLM orchestration, deterministic-probabilistic composition, workflow specification, symbolic verification, structured prompt language

2 Introduction

2.1 The Problem: Fragmentation in LLM Programming

Building LLM-powered workflow systems today demands a daunting intersection of skills. A practitioner must master prompt engineering for effective LLM interaction, Python programming for orchestration logic, API wrangling across disparate model providers, and manual state management for multi-step reasoning. Each popular framework — LangGraph [1], AutoGen [2], CrewAI [3] — introduces its own abstractions, API surfaces, and execution models, creating a fragmented ecosystem where knowledge transfers poorly and vendor lock-in is the norm.

This situation echoes a pattern the data industry has seen before. In the early days of relational databases, programmers wrote imperative C and COBOL code to traverse data structures, manage cursors, and handle errors — all in application-specific ways. The introduction of SQL in the 1970s transformed this landscape by providing a declarative specification layer: users described *what* data they wanted, not *how* to retrieve it.

This separation enabled decades of optimizer innovation underneath a stable surface language. The current state of LLM programming is analogous to the pre-SQL era: powerful capabilities buried under layers of imperative glue code.

The barrier to entry compounds the problem. To build even a simple self-refining workflow — one that drafts, critiques, and revises its own output — a developer must write 80–150 lines of Python across multiple framework abstractions. This excludes the vast population of domain experts, analysts, and researchers who understand their workflows but lack the software engineering skills to express them in imperative orchestration code.

2.2 The Two-Mode Gap

All existing frameworks — declarative and imperative alike — operate in a single computation mode. LLM-centric systems (LangGraph, DSPy, AutoGen) produce approximate, non-reproducible outputs shaped by the model’s training distribution. Symbolic tools (SymPy, SageMath, Lean) produce exact, reproducible, machine-verifiable results. Neither world speaks the other’s language within a unified programming model. We call this the **two-mode gap**.

Real agentic tasks routinely require both. A homework assistant must plan a solution in natural language (probabilistic), execute each algebraic step exactly (deterministic), verify the result (deterministic), then explain it to the student (probabilistic). The coupling between modes may be *loose* — distinct phases passing data once — or *tight*: an **ASSERT** failure immediately triggers a corrective **GENERATE**, which revises the plan and re-enters the symbolic engine. Both patterns arise in practice; both require application-specific glue code to coordinate state transfer and error propagation at every mode crossing.

The gap is not an engineering inconvenience — it is an expressiveness ceiling. No existing workflow language lets the programmer declare “compute this step exactly” and “generate this step approximately” in the same specification. SPL closes it through *declarative composition*: the programmer specifies what each step computes and which mode it inhabits; the runtime resolves the boundary, manages state transfer, and enforces verification gates — without glue code.

The dual-process theory of human cognition [17] offers a useful analogy — but requires a critical correction before it applies here. Kahneman’s framing names the two systems *fast* and *slow*, conflating epistemic mode with cognitive speed. That conflation misleads in a computational setting. When physicists worked out quantum mechanics from first principles, the process was intuitive, exploratory, and probabilistic — and extraordinarily slow, unfolding over years of collective effort with no guaranteed convergence. Once Schrödinger’s equation was established, *solving* it became a deterministic, mechanical process — but one whose speed was entirely a property of the tool: slow by hand, fast by computer. The same deterministic mode; two very different speeds. Speed tracks the implementation, not the mode.

The relevant distinction is epistemic, not temporal. System 1 is *probabilistic and intuitive*: it pattern-matches, generates hypotheses, and produces approximate outputs shaped by prior experience — regardless of wall-clock time. System 2 is *deterministic and mechanical*: it applies explicit rules to produce exact, reproducible results — regardless of whether those rules execute in milliseconds or minutes. In SPL’s computational mapping: **GENERATE** and **EVALUATE** are System 1 primitives (LLM inference, probabilistic, quality-assessed but not guaranteed exact); **SOLVE** and **ASSERT** are System 2 primitives (symbolic kernel, deterministic, machine-verifiable). A SymPy kernel may compute a derivative faster than the LLM does; a formal Lean 4 proof may take longer. Neither fact changes which mode each step inhabits. Mode — not speed — is the organizing principle.

2.3 Our Contributions

We present SPL, a declarative language that composes both computation modes in a single coherent specification. The primary contributions of this paper are:

C1. Declarative two-mode composition. SPL provides **SOLVE** and **ASSERT** — primitives that route computation to a live Python kernel (IPython) rather than an LLM — alongside **GENERATE** and **EVALUATE**

for the probabilistic mode. A single declarative specification freely composes both modes; variable bindings flow between them through a shared `@variable` namespace without programmer-managed marshaling. The mode boundary is visible in the source and managed entirely by the runtime.

C2. The DODA principle. Design Once, Deploy Anywhere: a single `.spl` specification is the invariant. The choice of model provider (Ollama, OpenRouter, Anthropic, Momagrid) and verifier (SymPy, SageMath, Lean) is resolved at invocation time via CLI flags, not embedded in the specification. The same file runs on a developer’s laptop and on a distributed inference grid without modification — declarative composition is the mechanism that makes this possible.

C3. The verifier ladder. We formalize a three-rung hierarchy of symbolic verifiers — SymPy (algebraic), SageMath (mathematical), Lean 4 (formal proof) — as ascending correctness guarantees composable into any SPL workflow via `ASSERT`. Each rung is a Python-callable tool registered at runtime; the workflow specification is agnostic to which rung is active. The ladder is itself declared, not coded: the escalation logic is `ASSERT` chains in `.spl`, not Python conditionals.

C4. Controlled empirical evaluation. A 1200-run experiment ($10 \text{ models} \times 20 \text{ problems} \times 2 \text{ arms} \times 6 \text{ difficulty tiers} \times 3 \text{ repeats}$) on symbolic mathematics provides quantitative evidence of the two-mode composition benefit and surfaces a structured capability hierarchy: models that score 100% in the probabilistic arm (LLM-only math) may score 0% in the two-mode arm (solver), not because they cannot solve the math but because they cannot produce the structured decomposition the deterministic engine requires. This reveals that format-compliance for declarative composition is a separable capability from mathematical reasoning.

2.4 Paper Organization

Section 2 surveys related work, positioning SPL against imperative frameworks, declarative systems, and neurosymbolic AI. Section 3 presents the SPL language design — the full primitive set, semantics, and formal grammar. Section 4 formalizes the verifier ladder. Section 5 describes the implementation: how the two-mode executor dispatches probabilistic nodes to the adapter layer and deterministic nodes to the kernel layer, with model provider and verifier both selected at invocation time. Section 6 presents the 1200-run empirical evaluation ($r=3$). Section 7 discusses strengths, limitations, and future directions. Section 8 concludes.

3 Related Work

3.1 The Evolution of Data Programming

The history of data programming offers a clear precedent for the transition SPL proposes. In the 1970s, IBM’s System R project demonstrated that a declarative query language (SQL) could match or exceed the performance of hand-coded navigational database access, because the declarative form exposed optimization opportunities invisible in imperative code [4]. By the 1980s, SQL had become the industry standard, and Oracle’s PL/SQL extension demonstrated that procedural control flow (loops, variables, exception handling) could be layered on top of declarative queries without sacrificing the optimizer’s ability to reason about individual `SELECT` statements [5].

The parallel to LLM programming is direct. Today’s imperative frameworks — LangGraph’s state graphs, AutoGen’s conversation loops, CrewAI’s role-based agents — are the equivalent of pre-SQL navigational code: powerful, flexible, and impossibly fragmented. SPL proposes that the same declarative revolution can occur for LLM orchestration, following the same evolutionary arc: atomic declarative queries (`PROMPT`) compose into procedural workflows (`WORKFLOW`) that an optimizer can eventually rewrite and route without altering the specification.

A critical difference from the SQL era: data queries are deterministic. LLM orchestration is not. This demands new primitives — `EVALUATE` for semantic branching, `WHILE` for quality-gated iteration, `EXCEPTION` for probabilistic failure modes — that have no SQL equivalent. And it demands a second computation mode entirely: `SOLVE` and `ASSERT` for the cases where a result must be exact rather than approximate.

3.2 Imperative Orchestration Frameworks

LangGraph [1] models workflows as directed graphs with explicit state management. Nodes represent computation steps, edges represent transitions, and state is a mutable dictionary passed through the graph. The framework provides fine-grained control but requires significant Python expertise, offers no formal grammar, and has no optimizer. Integrating a symbolic step requires writing a custom node that manually calls the target library and handles its exceptions.

AutoGen [2] focuses on multi-agent conversation patterns where agents exchange messages. Its strength is natural dialogue flows; its limitation is that orchestration logic is embedded in Python and tightly coupled to the runtime.

CrewAI [3] introduces a role-based metaphor where specialized agents collaborate. Intuitive for team-like workflows; Python-first with no path to compilation or optimization.

PydanticAI takes a code-first approach, prioritizing developer ergonomics and type safety. Like the above, it operates exclusively in the probabilistic mode with no symbolic integration layer.

None of these frameworks provides a language-level mechanism to say “execute this step deterministically and verify the result before the workflow proceeds.”

3.3 Declarative LLM Systems

LMQL [6] (ETH Zurich, 2022) pioneered SQL-like LLM interaction with output constraints (type, length, regex), achieving 26–85% cost reduction through constrained decoding. LMQL operates at the query level — individual prompts with output validation — and does not provide constructs for multi-step workflows or deterministic integration.

DSpy [7] (Stanford, 2023) introduces declarative modules (**Predict**, **ChainOfThought**) with a compiler that automatically optimizes prompts via few-shot bootstrapping. DSPy’s “declarative” is at the module-interface level: it optimizes which prompts to use, not how to orchestrate multi-step workflows. It provides no iteration, no exception handling, no symbolic integration, and no formal grammar.

SGLang [8] (UC Berkeley, 2023) provides structured generation with runtime optimizations (RadixAttention, up to 6.4× throughput). SGLang optimizes inference execution below the level of agentic patterns; it does not address workflow orchestration.

Agent Spec [9] (Oracle Labs, 2025) defines a YAML/JSON schema for framework-agnostic agent specification. A configuration format, not an executable language: it lacks control flow, iteration, and exception handling within the specification itself.

eBay DSL [10] (December 2025) separates workflow specification from implementation across multiple backend languages, achieving 60% reduction in development time. Focused on enterprise pipeline configuration; does not formalize LLM-specific primitives or symbolic integration.

Agentics 2.0 [11] (IBM, March 2026) introduces a logical transduction algebra where LLM calls are typed semantic transformations composable via algebraically grounded operators. The closest concurrent work in formalism, but algebraic rather than SQL-inspired, and still operating exclusively in the probabilistic mode.

3.4 Neurosymbolic AI

A growing body of work combines neural and symbolic computation, but at the model or algorithm level rather than the workflow language level.

AlphaGeometry [12] (DeepMind, 2024) solves Olympiad geometry problems by alternating a neural language model (generating proof steps) with a symbolic deduction engine (verifying them). The neuro-symbolic loop is hard-coded to the geometry domain and implemented as a custom Python pipeline. SPL generalizes this pattern to arbitrary domains: the **SOLVE/ASSERT** primitives express any such loop declaratively, and the verifier (geometry engine, SymPy, Lean) is a pluggable runtime choice rather than a hard-coded dependency.

Neural theorem proving systems (PACT, LeanDojo [13], HyperTree Proof Search [14]) use LLMs to generate Lean or Coq proof tactics, with the proof assistant as the verifier. SPL’s verifier ladder places Lean at rung R3 — the highest correctness guarantee — accessible from any workflow via **ASSERT**.

Program synthesis systems (Codex [15], AlphaCode [16]) generate code and execute it to verify functional correctness. SPL’s **SOLVE** construct is the declarative equivalent of this loop: the programmer specifies the expression template; the kernel executes it; **ASSERT** verifies the result — without the programmer writing the execution and verification harness in Python.

The common thread across these systems is that neuro-symbolic integration is implemented as bespoke application logic rather than as a language primitive. SPL’s contribution is to make that integration a first-class, reusable construct expressible in few keywords.

3.5 Positioning Summary

Table 1: System Comparison

System	Paradigm	Grammar	Semantic Eval	Sym. Integration	Multi-Runtime
LangGraph	Imperative	No	No	Manual (Python)	No
AutoGen	Imperative	No	No	Manual (Python)	No
CrewAI	Imperative	No	No	Manual (Python)	No
LMQL	Declarative	Yes	No	No	No
DSPy	Declarative	No	No	No	No
SGLang	Declarative	Yes	No	No	No
Agent Spec	Declarative	Yes (schema)	No	No	Partial
eBay DSL	Declarative	Yes	No	No	Yes
Agentics	Algebraic	Yes (algebra)	No	No	No
AlphaGeometry	Neuro-sym.	No	No	Yes (domain-specific)	No
SPL	Declarative	Yes (EBNF)	Yes	Yes (pluggable)	Yes (14 adapters)

SPL is the only system that provides a formal executable grammar, LLM-as-judge semantic evaluation, and *declarative* deterministic-probabilistic composition via **SOLVE/ASSERT** alongside **GENERATE/EVALUATE** — with a unified adapter layer spanning local, cloud, and decentralized execution. The critical distinction from every other declarative system in the table is that SPL’s declarativity extends across the mode boundary: the `.spl` source specifies composition of both modes; no Python glue bridges them.

4 The SPL Language

4.1 Design Principles

SPL is governed by the following design principles:

P1. Declarative composition over imperative implementation. The programmer specifies *what* each step computes and *which mode* it inhabits — not which model, which verifier, or which execution path achieves it. The two modes may be woven loosely (distinct probabilistic and deterministic phases passing data between them) or tightly (fine-grained interleaving where **ASSERT** failure immediately re-enters **GENERATE**); SPL handles both ends of this spectrum with the same primitives. The runtime resolves the mode boundary; the `.spl` source is the invariant across both coupling styles, both model providers, and both deployment environments.

P2. Two-mode primitives as co-equal first-class constructs. SPL provides primitives for both computation modes — System 1 (probabilistic, intuitive) and System 2 (deterministic, mechanical) in the dual-process sense [17], reinterpreted epistemically rather than temporally (see §1). The probabilistic mode (**GENERATE**, **EVALUATE**, **WHILE**, **EXCEPTION**) addresses the characteristics of LLM computation: approximate, non-reproducible, quality-assessed, cost-sensitive. The deterministic mode (**SOLVE**, **ASSERT**) addresses the characteristics of symbolic computation: exact, reproducible, machine-verifiable. Neither mode is subordinate to the other in the language design — their composition in a single specification is the defining property of SPL.

P3. Shared variable space. Outputs of **GENERATE** steps are available as inputs to **SOLVE** steps, and vice versa, through a single `@variable` namespace. The programmer does not marshal values between modes; the runtime manages the boundary.

P4. DODA — Design Once, Deploy Anywhere. The `.spl` file is the invariant. Physical decisions — which LLM, which verifier, which infrastructure — are resolved at invocation time via `--adapter`, `--model`, and `--kernel` flags. The same specification is the logical description of the workflow regardless of where it runs.

P5. Compilation target. The language is designed to produce an optimizable intermediate representation (IR), enabling future optimizer passes (**GENERATE** merging, **SELECT** caching, model routing) without changing the surface language.

4.2 The SQL Analogy

The mental model of SPL is grounded in its parallel to the SQL and PL/SQL stack:

- **SELECT** assembles the context that flows *into* the LLM — analogous to `SELECT ... FROM ... WHERE` gathering rows from a table.
- **GENERATE** invokes the LLM and captures what flows *out* — analogous to the result set returned by a SQL query.
- **PROMPT** binds **SELECT** and **GENERATE** into a named, independently executable unit — analogous to a single SQL query.
- **WORKFLOW** adds variables, loops, branching, and exception handling around **PROMPT** calls — analogous to PL/SQL layering procedural control flow over SQL queries.
- **SOLVE** routes a Python expression to the deterministic kernel — an extension that has no SQL analogue.

This last point marks the boundary of the SQL analogy. SPL must go further because LLM orchestration requires a second computation mode that relational data access does not.

4.3 Probabilistic Primitives

4.3.1 SELECT and GENERATE — The Atomic I/O Contract

SELECT assembles the prompt from multiple context sources; **GENERATE** invokes the LLM and captures the result:

```
WORKFLOW explain_concept
  INPUT: @concept TEXT, @audience TEXT DEFAULT 'undergraduate'
  OUTPUT: @explanation TEXT
DO
  GENERATE explanation(concept, audience) INTO @explanation
  SELECT
    system_role('You are a clear, precise instructor.'),
    @concept AS concept,
    @audience AS audience
```

```
PROMPT 'Explain @@concept@@ to a @@audience@@ student in 3 sentences.'
```

```
END
```

4.3.2 EVALUATE — Semantic Branching

EVALUATE uses the LLM as a runtime judge for conditions that cannot be expressed as deterministic comparisons:

```
EVALUATE @explanation
  WHEN contains('undefined') OR contains('undefined symbol') THEN
    @explanation := @explanation + '\n\n[Note: terms defined in appendix]'
  WHEN = 'satisfactory' THEN
    @status := 'complete'
  ELSE
    @status := 'needs_revision'
END
```

Quoted string conditions are dispatched to the LLM judge; equality comparisons (`= 'satisfactory'`) are evaluated deterministically. This hybrid detection rule means the programmer does not annotate which branches are semantic and which are deterministic — the parser infers it from syntax.

4.3.3 WHILE — Quality-Gated Iteration

```
WHILE 'explanation is incomplete or unclear' DO
  GENERATE refine_explanation(@explanation, @feedback) INTO @explanation
END
```

The termination condition is evaluated by the LLM on each iteration: the executor assembles all current `@variable` values as context and asks the LLM judge whether the condition still holds. A built-in maximum iteration guard (configurable, default 5) prevents infinite loops.

4.3.4 EXCEPTION — LLM-Aware Error Handling

```
EXCEPTION
  WHEN HallucinationDetected THEN
    RETURN @result WITH status = 'hallucination_detected'
  WHEN ContextLengthExceeded THEN
    @problem := summarize(@problem)
    RETRY
  WHEN BudgetExceeded THEN
    RETURN @partial_result WITH status = 'budget_exceeded'
END
```

The exception taxonomy formalizes failure modes specific to LLM computation — hallucination, refusal, context overflow, budget violation — that have no equivalent in conventional exception hierarchies.

4.4 Deterministic Primitives

4.4.1 SOLVE — Kernel-Routed Computation

SOLVE dispatches a Python expression to the live IPython kernel and assigns the result to a workflow variable:

```
SOLVE @derivative SYMPY := "diff(@@expression@@, x)"
```

The `@@variable@@` template syntax binds any `@variable` currently in scope into the expression before dispatch. This is the boundary-crossing mechanism: the LLM's output (held in `@expression`) becomes

the input to the symbolic engine without programmer-managed marshaling. The result (`@derivative`) is available immediately to subsequent `GENERATE` or `SOLVE` steps.

`SOLVE` is mode-explicit by design. The programmer cannot accidentally invoke the symbolic engine from a `GENERATE` call or the LLM from a `SOLVE` call. The mode boundary is visible in the source.

4.4.2 ASSERT — Verification Gate

`ASSERT` evaluates a Boolean expression in the kernel and gates workflow continuation on the result:

```
ASSERT simplify(@@sympy_result@@ - diff(@@expression@@, x)) == 0
    OTHERWISE RETURN @explanation WITH status = 'verification_failed'
```

On success, the workflow continues — the claim is machine-verified. On failure, the `OTHERWISE` body executes, typically committing a result with an error status or escalating to a higher verifier rung. `ASSERT` is the mechanism by which SPL workflows achieve correctness guarantees: a step that passes `ASSERT` is not merely plausible but mathematically proven correct by the kernel.

4.4.3 CALL PARALLEL — Concurrent Branch Dispatch

```
CALL PARALLEL
    CALL arm_solver(@problem) INTO @result_solver
    CALL arm_llm_only(@problem) INTO @result_llm
END
```

Branches execute concurrently via `asyncio.gather` on a single node, or route to distinct Momagrid worker nodes on a distributed grid. This construct makes A/B experiments — deterministic arm vs. probabilistic arm — a language-level primitive rather than a test-harness concern.

4.5 The Two-Mode Composition Pattern

Real agentic tasks follow a recurring four-stage lifecycle: **PLAN** (System 1 decomposes the problem), **SOLVE** (System 2 executes it exactly), **ASSERT** (System 2 verifies the result), **EXPLAIN** (System 1 narrates it in natural language). The following example shows this lifecycle expressed declaratively in the following SPL workflow:

```
WORKFLOW solve_with_verification
    INPUT: @problem TEXT
    OUTPUT: @explanation TEXT
DO
    -- [PLAN --- System 1 / Probabilistic] LLM decomposes the problem
    GENERATE decomposition(@problem) INTO @steps_text
    SELECT @problem AS problem
    PROMPT 'Decompose this problem into symbolic computation steps,
           one per line in the format: expression|operation'

    -- [PLAN --- System 1 / Probabilistic] Sanity gate before kernel handoff
    GENERATE plan_check(@steps_text, @problem) INTO @plan_verdict
    SELECT @steps_text AS plan, @problem AS problem
    PROMPT 'Does this plan correctly address the problem? Reply: pass or fail'

    EVALUATE @plan_verdict
    WHEN = 'fail' THEN
        RETURN @explanation WITH status = 'plan_sanity_error'
END
```



```

-- [SOLVE --- System 2 / Deterministic] Kernel executes each symbolic step exactly
CALL solve_chain(@steps_text) INTO @verified_result

-- [ASSERT --- System 2 / Deterministic] Verify the chain answer against the problem
ASSERT verify(@@verified_result@@, @@problem@@)
    OTHERWISE RETURN @explanation WITH status = 'verification_failed'

-- [EXPLAIN --- System 1 / Probabilistic] LLM narrates the verified result
GENERATE explanation(@verified_result, @problem) INTO @explanation
    SELECT @verified_result AS result, @problem AS problem
    PROMPT 'Explain this verified solution in clear prose: @@result@'

RETURN @explanation WITH status = 'complete'
END

```

This 30-line workflow composes five mode transitions declaratively: System 1 decomposition $\rightarrow$ System 1 sanity gate $\rightarrow$ System 2 execution $\rightarrow$ System 2 verification $\rightarrow$ System 1 narration. Each transition is expressed in SPL syntax; no Python bridges the modes.

The `@@var@@` template syntax is the *formalization boundary* of SPL: the seam at which System 1 outputs (unstructured text from `GENERATE`) are cast into typed expressions consumed by System 2 (`SOLVE/ASSERT`). The programmer does not write serialization code; the runtime resolves variable bindings before kernel dispatch. The mode crossing is visible at the source level — every `@@`-delimited reference marks a System 1 $\rightarrow$ System 2 handoff — and is managed entirely by the language.

The programmer writes the same `.spl` file regardless of which LLM and which symbolic engine execute at runtime — the declarative specification is the invariant, and the DODA principle holds across both the model axis and the verifier axis.

4.6 Formal Grammar

The SPL grammar defines the following key production rules for two-mode integration:

```

solve_stmt    ::= 'SOLVE' '@' identifier ['SYMPY' | 'SAGE' | 'LEAN'] ':= ' string_expr
assert_stmt   ::= 'ASSERT' python_expr ['OTHERWISE' stmt_block]
parallel_stmt ::= 'CALL' 'PARALLEL' (call_stmt)+ 'END'
stmt          ::= ... | solve_stmt | assert_stmt | parallel_stmt

```

The `SYMPY | SAGE | LEAN` type annotation on `SOLVE` is optional; when omitted, the runtime selects the kernel registered at invocation time. The full grammar is provided in Appendix A.

5 The Verifier Ladder

5.1 Motivation: Ascending Correctness Guarantees

LLM-generated mathematical reasoning is probabilistic by nature: a model scoring 95% on a benchmark is wrong on 1 in 20 problems, with no signal to the caller about which ones. Symbolic verification closes this gap — but symbolic verification itself is not monolithic. A SymPy check of polynomial differentiation takes milliseconds and requires no external tooling. A formal Lean 4 proof of the same claim takes seconds, requires Mathlib, and produces a machine-checkable certificate that any Lean installation can re-verify independently. The two checks are not interchangeable: one is appropriate for routine computation, the other for publication-quality assertions.

The SPL verifier ladder makes this trade-off explicit and programmable. Three rungs address different cost/confidence operating points:

Rung Engine		What it certifies	Setup	Typical use
R1	SymPy (python3 kernel)	Algebraic identity, calculus, matrix operations	<code>pip install sympy</code>	Routine STEM computation
R2	SageMath (sagemath kernel)	Number theory, geometry, polynomial rings, combinatorics	<code>pip install 'spl-llm[sage]'</code>	Deeper mathematical claims
R3	Lean 4 (lean_bridge)	Machine-checked formal proof against Mathlib	<code>bash setup_lean.sh --with-mathlib</code>	Publication-quality assertions

A workflow selects a rung via the `--kernel` and `--param backend=` flags at invocation time. The `.spl` source is identical across all three rungs — the DODA principle applies to verifier selection just as it applies to model selection.

5.2 Rung R1: SymPy

The SymPy rung runs inside an IPython kernel started with the standard `python3` kernelspec. Seventeen mathematical operations are registered as SPL tools via `CREATE TOOL_API`:

```
CREATE TOOL_API solve_step_with_sympy(expression, operation)
'''
import sympy as sp
from sympy.abc import x, y, z, t, n
ops = {
    "diff":      lambda e: sp.diff(e, x),
    "integrate": lambda e: sp.integrate(e, x),
    "solve":     lambda e: sp.solve(e, x),
    "eigenvalues": lambda e: sp.Matrix(eval(e)).eigenvals(),
    "dsolve":    lambda e: sp.dsolve(sp.sympify(e)),
    ...    -- 12 further operations
}
result = ops[operation](sp.sympify(expression))
return f"{result}|{sp.latex(result)}"
'''
```

The tool returns a `bare_result|human_readable` protocol: the bare result is available as a string for subsequent `@@var@@` template substitution; the human-readable side is written to the chain trace log. On failure the sentinel `solver_error|...` is returned, which the workflow detects via `EVALUATE` and exits with an explicit status rather than propagating a bad value downstream.

5.3 Rung R2: SageMath

SageMath extends SymPy’s algebraic reach into number theory (Galois groups, elliptic curves, modular arithmetic), differential geometry, and combinatorics. The runtime discovers the SageMath Jupyter kernelspec automatically and raises a clear install error if absent (see Appendix C for setup). The same `SOLVE / ASSERT` primitives and `@@var@@` template protocol work unchanged — the only runtime difference is which kernelspec launches. This is the DODA principle applied at the verifier level: the workflow programmer writes identical SPL regardless of which mathematical engine executes beneath.

5.4 Rung R3: Lean 4

The Lean rung addresses a qualitatively different class of claim: not “does this algebraic identity hold for generic symbols” but “is this mathematical statement a theorem provable from first principles within Mathlib.” SPL integrates Lean 4 via `spl3/lean_bridge.py`, which wraps the Lean REPL in a persistent session. The rung follows a three-stage protocol, each stage expressed in pure SPL: (1) **Formalize** — the LLM translates

the natural-language claim into a Lean 4 statement; (2) **Typecheck** — the kernel verifies it compiles against Mathlib, with an LLM-driven repair loop; (3) **Prove** — the LLM writes tactic proof code, which the kernel checks and repairs. The full SPL protocol is given in Appendix G.

The Lean rung produces a *badge* — `machine_proved`, `statement_checked`, `unfaithful`, or `unverified` — which becomes the workflow’s output status. A failed proof never blocks delivery: the claim is returned with its statement-checked grade, and the higher badge is withheld. This allows the Lean rung to run speculatively in a `CALL PARALLEL` branch while the primary answer is already delivered via the SymPy arm.

5.5 ASSERT as the Inter-Rung Gate

ASSERT is the mechanism by which the verifier ladder enforces correctness within a workflow. Each rung produces a value that can be checked, and ASSERT failure triggers escalation:

```
-- Try R1 first (inside a workflow body)
SOLVE @derivative SYMPY := "diff(@@expression@@, x)"
ASSERT simplify(@@derivative@@ - diff(@@expression@@, x)) == 0
  OTHERWISE
    -- R1 could not verify; escalate to R2
    SOLVE @derivative SAGE := "SR(@@expression@@).diff(x)"
    ASSERT bool(@@derivative@@ == diff(@@expression@@, x))
      OTHERWISE RETURN @explanation WITH status = 'r2_failed'
```

This escalation pattern is idiomatic SPL declarative composition: try the cheapest rung, gate on ASSERT, compose with a higher rung on failure. No Python glue code is required; the entire escalation logic — including the mode transitions between LLM narration and kernel verification — is expressed declaratively in the `.spl` source.

5.6 Pluggability

The verifier ladder is open-ended: any Python-callable verifier plugs in without modifying the language or runtime. A workflow author registers a new tool via `CREATE TOOL_API`, selects it via a parameter or `--kernel` flag, and writes ASSERT conditions against its output. Beyond the three rungs demonstrated in this paper, the pattern generalizes to: unit verification (pint), graph property checking (NetworkX), constraint solving (Z3), type checking (mypy), or statistical hypothesis testing (scipy.stats). The language makes no assumption about what the kernel computes — only that it is Python-callable and that the result is a string the workflow can inspect.

6 Implementation

The executor parses a `.spl` source into an AST and dispatches each node to one of two engines: probabilistic nodes (`GENERATE`, `EVALUATE`, `WHILE`, `EXCEPTION`) route to the adapter layer, which normalizes calls across 14 LLM providers behind a two-method interface (`generate` / `generate_multimodal`). Deterministic nodes (`SOLVE`, `ASSERT`) route to a persistent kernel session — an out-of-process Jupyter kernel for symbolic computation, or a lightweight in-process substrate for tool execution. The kernel path activates only when `SOLVE` or `ASSERT` is encountered; otherwise no kernel is started. Adapter and kernel are both selected at invocation time (`--adapter`, `--model`, `--kernel`) and never named in the `.spl` source — the deterministic mode of DODA mirrors the probabilistic one.

The two modes share state through `@@var@@` template substitution: before a `SOLVE` expression is dispatched to the kernel, the executor inlines current SPL variable values into the expression string. The result is captured and bound back to a `@var`, making it available to subsequent `GENERATE` or `EVALUATE` nodes. This variable-passing contract is the integration seam between the two modes; no explicit marshalling code is required in the workflow.

The same `.spl` source compiles to idiomatic code in four target frameworks via `spl3 splc` (`--target langgraph, go, typescript, pocketflow`). The compiler reads the AST directly and maps each construct to the target’s natural idiom — `StateGraph` nodes for LangGraph, goroutines for Go parallel branches, `Promise.all` for TypeScript. Full runtime details (kernel substrates, adapter bootstrap, template resolution, compilation targets, ecosystem tooling) are provided in Appendices B, F, and G.

7 Empirical Evaluation

7.1 Experiment Design

We evaluate SPL’s deterministic integration using a controlled dual-arm experiment on symbolic mathematics. The benchmark consists of 20 problems spanning six difficulty tiers (T0–T5), split across two symbolic backends:

Tier	Backend	Category	Example
T0	SymPy	Single-step polynomial	$d/dx (x^4 - 2x^2 + 1)$
T1	SymPy	Multi-step polynomial chains	<code>expand</code> $\rightarrow$ <code>factor</code> $\rightarrow$ <code>solve</code>
T2	SymPy	Transcendental / limits / series / trig	$\sin(x)/x$ as $x \rightarrow 0$; Taylor $\sin(x)$ deg 5
T3	Sage	Integration / linear systems / eigenvalues	$\int \sqrt{4 - x^2} dx$; eigenvalues of $[[1, 2], [3, 4]]$
T4	Sage	Laplace transforms / ODEs / summation / roots	$\mathcal{L}\{e^{-2t}\}$; $y' = y$, $y(0) = 1$; $\sum 1/n^2$
T5	Sage	Expert ODE + transform verification	$y'' - 3y' + 2y = 0$; $\mathcal{L}^{-1}\{s/(s^2 + 4)\} + \text{verify}$

T0–T2 problems (10 total) route to the SymPy kernel; T3–T5 (10 total) route to the SageMath kernel. The backend is a per-problem parameter; the `.spl` workflow is identical across both.

Ten models were evaluated: `sonnet-4-6`, `gemma3`, `gemma4:e2b`, `qwen2.5`, `deepseek-v2:16b`, `phi3`, `phi4`, `llama3.2`, `l1m2.5`, and `rnj-1`. Models with mandatory extended chain-of-thought (`qwen3`, `deepseek-r1`) were excluded: the thinking trace exhausts the token budget before any structured output is emitted, making them fundamentally incompatible with the `expr|op` contract (see Appendix I). Each of the ten models ran all 20 problems under two arms:

- **Solver arm** (`enable_solver=true`): the model decomposes the problem into `expr|op` steps; the symbolic kernel (SymPy or Sage) executes each step exactly; the verified chain is fed back to the model for narration. Pass = status `complete` (all steps kernel-verified end-to-end).
- **LLM-only arm** (`enable_solver=false`): the model solves and narrates directly with no kernel involvement. Pass = non-empty response returned (status `complete` or `unverified_success`). Mathematical correctness is not verified in this arm.

Two sessions were run: a pilot (`exp-20260615-073849`, `r=1`, 400 cells) and a repeated run (`exp-20260615-191224`, `r=3`, 1200 cells). Results below are from the `r=3` session; the pilot is retained in Appendix E for comparison.

7.2 Results

Pass rate and latency by model and arm (mean over 3 runs/cell, sorted by solver pass rate):

Model	Pass rate (%)			Latency		
	LLM-only	Solver	Δ	LLM-only	Solver	Δ
<code>gemma4:e2b</code>	97	93	−3	11.9s	14.8s	+25%
<code>sonnet-4-6</code>	100	85	−15	13.1s	9.7s	−25%
<code>rnj-1</code>	100	82	−18	7.4s	6.4s	−13%
<code>gemma3</code>	100	73	−27	4.7s	4.6s	−1%
<code>qwen2.5</code>	100	72	−28	8.6s	3.9s	−55%
<code>phi4</code>	100	67	−33	27.3s	12.1s	−56%
<code>llama3.2</code>	100	65	−35	5.0s	3.0s	−41%
<code>deepseek-v2:16b</code>	100	53	−47	16.8s	7.9s	−53%
<code>lfn2.5</code>	77	37	−40	7.1s	12.2s	+71%
<code>phi3</code>	100	32	−68	7.4s	4.1s	−45%

LLM-only “pass” = non-empty response (unverified). Solver “pass” = complete symbolic chain (machine-verified by SymPy or Sage kernel). The two arms measure different things: one measures output production, the other measures verified correctness.

7.3 Findings

F1 — Output-fence cleaning eliminated `plan_format_error` entirely. In a preliminary run, `phi4` produced its decomposition plans wrapped in markdown fences (````plaintext ... ````), causing `SyntaxErrors` in the kernel and 0% solver pass. Adding `strip_fences()` — a regex-based fence stripper applied immediately after `decompose_problem()` — resolved this. Across both the 400-cell pilot and the 1200-cell repeated run, zero failures were attributed to `plan_format_error` in any model. The dominant failure mode is `solver_error` (kernel-rejected expressions): format compliance is not the capability bottleneck — expression correctness at the kernel boundary is.

F2 — `solver_error` is the primary failure mode; `lfn2.5` is the `plan_error` outlier. Every failure in the solver arm is `solver_error` or `plan_error` — never `plan_format_error`. Models that fail are generating syntactically valid `expr` plans but producing expressions the kernel cannot evaluate: wrong variable names, unsupported operations for the active backend, or malformed expression syntax (e.g., `^` instead of `**`). `lfn2.5` is the outlier: 26/60 solver failures are `plan_error` (invalid step structure), not `solver_error`, indicating a different failure mode — plan generation rather than expression evaluation.

F3 — `gemma4:e2b` achieves near-solver parity; `sonnet-4-6` is the most stable model. `gemma4:e2b` scores 93% solver vs 97% LLM-only — the closest any model comes to zero-cost verified correctness. `sonnet-4-6` scores exactly 85% in both the `r=1` pilot and the `r=3` repeated run, confirming its result is stable. `rnj-1` drops from 90% (pilot) to 82% (repeated), revealing pilot variance. `phi4` shows the largest pilot inflation: 85% (`r=1`) → 67% (`r=3`), an 18-point drop that demonstrates why repeated runs are necessary for ranking stability.

F4 — Sage problems are structurally harder for the solver arm. SymPy problems (T0–T2): 78% aggregate solver pass rate. Sage problems (T3–T5): 54% aggregate solver pass rate. The Sage backend exposes harder decomposition demands: Laplace transforms, first- and second-order ODEs, infinite sums, and inverse transform verification require the model to correctly name Sage-specific operations (`laplace`, `inverse_laplace`, `solve_system`) and format expressions the Sage kernel can parse. T4 (50%) and T5 (43%) are the hardest tiers across all models.

F5 — Solver arm accelerates most models; high-accuracy models pay overhead. Seven models are faster in the solver arm, led by `qwen2.5` (−55%), `phi4` (−56%), `deepseek-v2:16b` (−53%). The kernel short-circuits LLM chain-of-thought: once decomposition is done, the symbolic engine resolves the chain without further LLM calls, and early-exit on `solver_error` skips the narration call entirely. `gemma4:e2b` (+25%) and `lfn2.5` (+71%) are slower — `gemma4:e2b` because high accuracy means it completes chains and pays the full narration call; `lfn2.5` because its 26 `plan_errors` require longer retry attempts before failing.

The SPL status codes (`plan_error`, `solver_error`, `complete`) make per-category failure analysis automatic — each cell records its failure mode, enabling per-tier, per-backend, and per-model breakdown without log parsing.

7.4 Discussion of Evaluation Design

Pass/fail criterion. The solver arm pass criterion is machine-verified: status `complete` means the symbolic kernel executed every decomposed step without error, producing a chain-verified result. The LLM-only arm pass criterion is weaker: status `complete` or `unverified_success` means the model produced a non-empty response. Mathematical correctness in the LLM-only arm is not verified. This asymmetry is deliberate — the comparison measures verified solver output against unverified LLM output, making the solver’s correctness guarantee the primary claim rather than a relative accuracy delta.

Repeated runs and ranking stability. The $r=1$ pilot (400 cells) and $r=3$ session (1200 cells) tell a consistent story at the top and bottom of the ranking but diverge in the middle. The top-3 ranking (gemma4:e2b $\rightarrow$ sonnet-4-6 $\rightarrow$ rnj-1) and bottom-2 (lfn2.5, phi3) are stable across both sessions. The middle tier shows pilot variance of up to 18 percentage points (phi4: 85% $\rightarrow$ 67%), confirming that $r=3$ is the minimum for reliable ranking. Full per-run comparison is in Appendix E.5.

Thinking-mode exclusion. Two model families were evaluated and excluded pre-experiment: `qwen3` and `deepseek-r1` run mandatory extended chain-of-thought that exhausts the 1500-token output budget before emitting any structured output. Both are fundamentally incompatible with the `expr|op` structured-output contract; the 10-model experiment uses non-thinking alternatives (`deepseek-v2:16b`, `llama3.2`) instead.

The experiment demonstrates the primary TMLR claim at scale: a single `.spl` workflow ran all 1200 cells — both arms, six tiers, two backends, three repeats — without any test-harness code. The dual-arm structure, status codes, backend routing, and per-run metadata were all features of the SPL workflow itself.

8 Discussion

8.1 The 2×2 Matrix Closed

Table 2 situates SPL against existing systems on the computation matrix from Section 1:

System	Probabilistic (design)	Probabilistic (runtime)	Deterministic (design)	Deterministic (runtime)
LangGraph	✓	✓	—	—
AutoGen	✓	✓	—	—
DSPy	✓	✓	—	—
AlphaGeometry	—	✓	✓	✓ (geometry only)
SPL	✓	✓	✓	✓ (any domain)

No prior workflow language fills all four cells for an arbitrary domain. AlphaGeometry [12] fills all four but is domain-locked to Euclidean geometry. SPL fills all four for any domain where a Python-callable verifier exists — and makes the boundary between cells *declarative* in the source language rather than implicit in framework internals.

The central consequence is that the programmer writes one `.spl` file and the runtime resolves which cells are active at invocation time. This is not a capability difference of degree but of kind: declarative composition means the `.spl` specification is the invariant across both dimensions — model provider and verifier rung — and correctness-vs-speed trade-offs are deployment decisions made at the command line, not embedded in the source.

8.2 The System 1/2 Decomposition

SPL’s two-mode architecture maps cleanly onto dual-process theory [17], with the epistemic reinterpretation established in §1: the distinction is *probabilistic/intuitive* versus *deterministic/mechanical*, not fast versus slow. System 1 (LLM, **GENERATE/EVALUATE**) excels at the parts of a task that require intuitive pattern recognition — reading a problem statement, choosing an approach, narrating a result in natural language. System 2 (kernel, **SOLVE/ASSERT**) handles the parts that require mechanical exactness — executing an algebraic step, checking a formal identity, certifying a proof. Which mode is faster depends entirely on the tool and problem; the mode assignment in the workflow does not.

The empirical findings confirm this decomposition in a concrete domain. Finding F1 is precisely a System 1/2 boundary failure: phi3, phi4, and gemma3 have strong System 1 capability (100% LLM-only math) but cannot perform the System 1 task SPL *actually* requires of them — translating a problem statement into a structured `expr|op` plan for System 2 to consume. This is not a System 2 failure (they never reach the kernel); it is a System 1 format-mapping failure at the formalization boundary. The capability hierarchy revealed by the experiment is: System 1 reasoning ability → System 1 format compliance → System 2 pipeline participation.

This framing motivates the format-compliance routing proposed in Future Work: a lightweight System 1 probe that checks whether the active model can satisfy the formalization boundary before committing to the solver arm.

8.3 The Structured Output Bottleneck

Finding F1 has a broader implication beyond symbolic mathematics. The solver arm exposes a structured output contract — a specific schema the LLM must satisfy to participate in a verified pipeline. Models that cannot satisfy this contract lose access to the verifier’s guarantees entirely, regardless of their underlying reasoning ability. This suggests a capability hierarchy that is not captured by existing benchmarks: mathematical reasoning ability → structured output compliance → verifier pipeline participation.

This hierarchy motivates a future SPL feature: an automatic model routing gate that probes format compliance before committing a model to the solver arm, falling back to the LLM-only arm for incapable models rather than recording plan failures. The **EVALUATE** primitive already provides the conditional dispatch mechanism; the probe itself would be a one-shot format compliance check as a **GENERATE** step before the main decomposition.

8.4 Pluggability vs. AlphaGeometry-Style Integration

AlphaGeometry [12] achieves state-of-the-art geometry theorem proving by hard-coding the neuro-symbolic loop: the neural model generates proof candidates, the symbolic engine (DD-Geometry) verifies them, and the system iterates. This tight coupling is appropriate for a single-domain system optimized for competition performance.

SPL takes the opposite design stance: the symbolic component is always an external, pluggable tool registered via `CREATE TOOL_API` or `@spl_tool`. This generality trades peak single-domain performance for domain-independence. A domain expert adds a new verifier in Python; the SPL workflow language and runtime require no modification. The same **ASSERT** semantics that check a SymPy polynomial identity can check a NetworkX graph property, a Z3 satisfiability result, or a scipy hypothesis test — any predicate the domain expert can express in Python.

8.5 Limitations

Kernel startup overhead. The IPython kernel adds ~2 seconds of cold-start latency per run. For short workflows with a single **SOLVE** step, this overhead is disproportionate. The `KernelSession` substrate (in-process `exec()`) is a lower-overhead alternative but lacks the full scientific Python environment of the Jupyter kernel.

Verifier ladder setup cost. R2 (SageMath) and R3 (Lean 4) require non-trivial installation beyond `pip install spl-llm`. SageMath requires a separate conda package or system install; Lean 4 requires the Elan toolchain and Mathlib download (~2 GB). These are not barriers for a practitioner setting up a proof-carrying pipeline, but they mean the full verifier ladder is not available in a zero-configuration deployment.

LLM-as-judge evaluation. The 400-run experiment uses an LLM sanity gate as the correctness oracle. For problems where the judge model is also uncertain, this introduces noise in both directions. Replacing the judge with a deterministic SymPy ground-truth check (where applicable) would give a cleaner signal and is planned for the next experiment round.

8.6 Future Work

Format-compliance routing. A pre-flight GENERATE step that probes whether the active model can emit the required format, gating the solver arm accordingly, would eliminate the plan-format-error class entirely.

Query optimizer. SPL’s declarative semantics open the door to a query-optimizer-style layer: merging redundant GENERATE calls, caching SELECT results, downgrading to a smaller model for cheap sub-tasks. None of this requires changes to the `.spl` source — it is a runtime optimization over the AST.

Proof-carrying workflows. R3 (Lean 4) currently generates a proof badge but does not embed the proof artifact in the workflow’s output. A proof-carrying extension would attach the Lean 4 certificate as a typed field in the RETURN payload, enabling downstream consumers to independently re-verify without re-running the workflow.

Distributed verifier routing. The Momagrid adapter [18] currently routes LLM calls to worker nodes. The same routing mechanism could route SOLVE dispatches to verifier-specialized nodes (a node with a GPU-accelerated SageMath install, or a node with Lean 4 and Mathlib pre-warmed), enabling a distributed verifier ladder over the existing Momagrid infrastructure.

9 Conclusion

Existing orchestration frameworks handle the probabilistic mode — LLM calls, multi-agent routing, chain-of-thought scaffolding — with increasing sophistication. They do not handle the deterministic mode: the closed-form computation, the algebraic check, the formal proof. Composing the two today requires custom Python glue that binds a particular model to a particular verifier in a non-portable, non-reusable, and non-declarative way.

SPL addresses this gap through *declarative composition*: a single `.spl` specification expresses both modes — probabilistic and deterministic — in the same syntax, with the same variable bindings, routed by the runtime to the appropriate engine. The programmer declares what each step computes; the boundary between modes is managed by the language, not hand-coded at each transition. The same workflow that calls an LLM to decompose a problem calls SymPy to verify each step, gates continuation on `ASSERT`, and narrates the verified result — all without glue code.

A 1200-run empirical evaluation ($r=3$, 10 models, 6 difficulty tiers) confirms the primary claim: the most capable models achieve 82–93% machine-verified correctness in the solver arm — `gemma4:e2b` reaches 93% (vs 97% LLM-only), demonstrating near-zero-cost verified computation. The evaluation surfaces a structural finding orthogonal to model capability: format compliance for declarative composition is a separable skill — models that cannot emit the structured decomposition the deterministic engine expects lose access to its guarantees entirely, regardless of their underlying mathematical reasoning ability. This motivates automatic compliance routing as a near-term extension.

The verifier ladder — SymPy for algebraic identity, SageMath for deeper mathematics, Lean 4 for formal proof — makes the correctness guarantee a runtime parameter, selectable at the command line without modifying the workflow specification. Any Python-callable oracle plugs in as a new rung. The DODA principle holds across both modes: model provider and verifier rung are deployment decisions, not specification decisions. SPL is released as `spl-llm` on PyPI, with a recipe-rich cookbook demonstrating declarative composition

from basic prompt chaining to formal proof verification. The source code and data are publicly available at <https://github.com/digital-duck/SPL.py>.

10 References

- [1] Z. Liu et al., “LangGraph: Building Stateful Multi-Agent Applications,” 2024.
- [2] Q. Wu et al., “AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation,” ICLR 2024.
- [3] J. Moura, “CrewAI: Framework for Orchestrating Role-Playing Autonomous AI Agents,” 2024.
- [4] M. Stonebraker and G. Kemnitz, “The POSTGRES Next Generation Database Management System,” CACM, 1991.
- [5] Oracle Corporation, “PL/SQL User’s Guide and Reference,” 1992.
- [6] L. Beurer-Kellner et al., “Prompting Is Programming: A Query Language for Large Language Models,” PLDI 2023.
- [7] O. Khattab et al., “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines,” ICLR 2024.
- [8] L. Zheng et al., “SGLang: Efficient Execution of Structured Language Model Programs,” NeurIPS 2024.
- [9] Oracle Labs, “Agent Spec: A Framework-Agnostic Agent Specification Schema,” 2025.
- [10] eBay Inc., “A Declarative DSL for Agent Workflow Specification,” 2025.
- [11] IBM Research, “Agentics 2.0: Logical Transduction Algebra for LLM Workflows,” 2026.
- [12] T. Trinh et al., “Solving Olympiad Geometry Without Human Demonstrations,” Nature, 2024.
- [13] K. Yang et al., “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models,” NeurIPS 2023.
- [14] G. Lample et al., “HyperTree Proof Search for Neural Theorem Proving,” NeurIPS 2022.
- [15] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv 2021.
- [16] Y. Li et al., “Competition-Level Code Generation with AlphaCode,” Science 2022.
- [17] D. Kahneman, “Thinking, Fast and Slow,” Farrar, Straus and Giroux, 2011.
- [18] Wen G. Gong, “Momagrid: A Decentralized Inference Grid for LLM Workflows,” unpublished manuscript, 2025. GitHub: <https://github.com/digital-duck/momagrid>. PDF: <https://github.com/digital-duck/dd-work/blob/main/docs/Momagrid-arxiv.pdf>

Appendix A: Complete EBNF Grammar

The grammar below covers the full SPL 3.0 surface language. Terminal symbols are quoted strings; non-terminals are lowercase identifiers. * = zero or more, + = one or more, ? = optional, | = alternation. The @@var@@ template syntax (used inside string expressions for kernel dispatch) is handled at the lexer level and is not shown separately.

```
(* Top-level structure *)
program      ::= (import_stmt | workflow_def | procedure_def
                  | create_fn_stmt | create_tool_stmt)*

import_stmt  ::= 'IMPORT' string_literal

workflow_def ::= 'WORKFLOW' identifier
               input_decl? output_decl? 'DO'
               stmt_list
               'END'

procedure_def ::= 'PROCEDURE' identifier
                input_decl? output_decl? 'DO'
                stmt_list
                'END'

input_decl   ::= 'INPUT' ':' param_spec (',' param_spec)*
output_decl  ::= 'OUTPUT' ':' param_spec (',' param_spec)*
param_spec   ::= '@'identifier type_ann? ('DEFAULT' expr)?
type_ann     ::= 'TEXT' | 'LIST' | 'SET' | 'INT' | 'FLOAT' | 'BOOL'
               | 'IMAGE' | 'AUDIO' | 'VIDEO'

(* Statements *)
stmt_list    ::= stmt*
stmt         ::= generate_stmt
               | evaluate_stmt
               | while_stmt
               | exception_stmt
               | solve_stmt
               | assert_stmt
               | call_stmt
               | call_parallel_stmt
               | return_stmt
               | logging_stmt
               | assign_stmt
               | if_stmt

(* Probabilistic primitives *)
generate_stmt ::= 'GENERATE' call_expr 'INTO' '@'identifier
                 select_clause? prompt_clause?

select_clause ::= 'SELECT' select_item (',' select_item)*
select_item   ::= system_role_call
               | '@'identifier 'AS' identifier
               | string_literal 'AS' identifier
               | rag_call
               | memory_call
```

```

prompt_clause    ::= 'PROMPT' string_literal

system_role_call ::= 'system_role' '(' string_literal ')'
rag_call         ::= 'rag' '(' string_literal (',' expr)? ')'
memory_call      ::= 'memory' '(' string_literal ')'

evaluate_stmt    ::= 'EVALUATE' '@' identifier
                  when_clause+ else_clause? 'END'
when_clause      ::= 'WHEN' condition 'THEN' stmt_list
else_clause      ::= 'ELSE' stmt_list
condition        ::= '=' string_literal          (* deterministic equality *)
                  | string_literal              (* semantic / LLM-judge *)
                  | 'contains' '(' string_literal ')'
                  | 'startswith' '(' string_literal ')'
                  | python_expr                 (* deterministic Python bool *)

while_stmt       ::= 'WHILE' condition 'DO' stmt_list 'END'

exception_stmt   ::= 'EXCEPTION' exception_clause+ 'END'
exception_clause ::= 'WHEN' exception_type 'THEN' stmt_list
exception_type   ::= 'HallucinationDetected' | 'ContextLengthExceeded'
                  | 'BudgetExceeded' | 'ModelRefused' | 'TimeoutError'
                  | 'WorkflowCompositionError' | identifier

(* Deterministic primitives *)
solve_stmt      ::= 'SOLVE' '@' identifier type_ann? kernel_hint? ':' python_expr
kernel_hint     ::= 'SYMPY' | 'SAGE' | 'LEAN' | 'PYTHON'

assert_stmt     ::= 'ASSERT' python_expr
                  ('OTHERWISE' stmt_list)?

(* Workflow composition *)
call_stmt       ::= 'CALL' call_expr 'INTO' '@' identifier
call_expr       ::= identifier '(' arg_list? ')'
arg_list        ::= arg (',' arg)*
arg             ::= '@' identifier | string_literal | named_arg
named_arg       ::= identifier '=' expr

call_parallel_stmt ::= 'CALL' 'PARALLEL' call_stmt+ 'END'

(* Output and side effects *)
return_stmt     ::= 'RETURN' expr
                  ('WITH' identifier '=' expr (',' identifier '=' expr)*)?
                  (* COMMIT is a deprecated alias for RETURN *)

logging_stmt    ::= 'LOGGING' expr ('LEVEL' log_level)?
log_level       ::= 'INFO' | 'DEBUG' | 'WARNING' | 'ERROR'

assign_stmt     ::= '@' identifier ':' expr

if_stmt         ::= 'IF' condition 'THEN' stmt_list
                  ('ELSE' stmt_list)? 'END'

```

```

(* Tool and function registration *)
create_fn_stmt  ::= 'CREATE' 'FUNCTION' identifier
                  '(' param_list? ')'
                  'RETURNS' type_ann
                  string_literal          (* Python body *)

create_tool_stmt ::= 'CREATE' 'TOOL_API' identifier
                  '(' param_list? ')'
                  string_literal          (* Python body *)

param_list      ::= param_spec (',' param_spec)*

(* Expressions *)
expr            ::= '@' identifier
                  | string_literal
                  | fstring_literal
                  | number_literal
                  | bool_literal
                  | none_literal
                  | list_literal
                  | map_literal
                  | binary_expr
                  | unary_expr
                  | call_expr

fstring_literal ::= 'f"' (char | '{' '@' identifier '}')* '"'
list_literal    ::= '[' (expr (',' expr)*)? ']'
map_literal     ::= '{' (string_literal ':' expr (',' string_literal ':' expr)*)? '}'
binary_expr     ::= expr ('+' | '-' | '*' | '/' | '==' | '!=' | '<' | '>' | '<=' | '>='
                        | 'AND' | 'OR' | '+' | '+') expr
unary_expr      ::= 'NOT' expr
python_expr     ::= string_literal          (* dispatched to kernel *)

(* Lexical *)
identifier      ::= [a-zA-Z_][a-zA-Z0-9_]*
string_literal  ::= '"' char* '"' | "'" char* "'"
number_literal  ::= [0-9]+ ('.' [0-9]+)?
bool_literal    ::= 'true' | 'false' | 'True' | 'False'
none_literal    ::= 'None' | 'null'
comment         ::= '--' char* newline

```

Comments (--) are line-scoped and stripped before parsing. String literals use double quotes by convention; single quotes are accepted but not recommended (parser quirk with the syntax highlighter). The @@var@@ interpolation in string literals sent to SOLVE/ASSERT is resolved by the executor before kernel dispatch.

Appendix B: Adapter System

B.1 Mandatory Adapters

Four adapters are always available after `pip install spl-llm`:

Adapter	Invocation	Protocol
ollama	<code>--adapter ollama --model gemma3</code>	HTTP POST to <code>localhost:11434/v1/chat/completions</code> (OpenAI-compatible)
claude_cli	<code>--adapter claude_cli</code>	Shells out to the claude CLI; captures stdout
openrouter	<code>--adapter openrouter --model qwen/qwen3-6b</code>	HTTPS to <code>api.openrouter.ai</code> ; requires <code>OPENROUTER_API_KEY</code>
momagrid	<code>--adapter momagrid</code>	POST <code>/tasks</code> → poll GET <code>/tasks/{id}</code> on <code>MOMAGRID_HUB_URL</code>

B.2 dd-llm Bridge (Cloud Providers)

When the `dd-llm` package is installed, ten additional cloud providers are available through a unified bridge adapter. The bridge normalizes provider-specific authentication and response shapes behind the same `BaseAdapter` interface:

Provider	Model prefix example
Anthropic	<code>anthropic/claude-sonnet-4-6</code>
OpenAI	<code>openai/gpt-4o</code>
Google	<code>google/gemini-2.0-flash</code>
Mistral	<code>mistral/mistral-large</code>
Cohere	<code>cohere/command-r-plus</code>
Together AI	<code>together/meta-llama/Llama-3-70b</code>
Groq	<code>groq/llama3-70b-8192</code>
Fireworks	<code>fireworks/accounts/fireworks/models/mixtral-8x22b</code>
Perplexity	<code>perplexity/sonar-large-32k-online</code>
DeepSeek	<code>deepseek/deepseek-chat</code>

B.3 Adapter Bootstrap Protocol

`spl3/adapters/__init__.py` resolves adapters in three-step priority order:

1. **dd-llm bridge** — if installed, registered first for all cloud model prefixes.
2. **Bespoke fallbacks** — `claude_cli` and `openrouter` registered as standalone adapters for environments without `dd-llm`.
3. **Always-available** — `ollama` and `momagrid` always registered last as unconditional fallbacks.

B.4 Two-Method Interface

All adapters implement:

```
class BaseAdapter:
    def generate(
        self, prompt: str, system: str, model: str, **kw
```

```
) -> GenerationResult: ...

def generate_multimodal(
    self, content: list[dict], system: str, model: str, **kw
) -> GenerationResult: ...
```

The executor dispatches to `generate_multimodal()` when any `INPUT` variable carries an `IMAGE`, `AUDIO`, or `VIDEO` type annotation; otherwise `generate()`. `GenerationResult` carries `content: str`, `model: str`, `latency_ms: float`, and `token_counts: dict`.

B.5 Momagrid Response Protocol

The Momagrid hub returns a nested result object that must not be flattened:

```
{
  "state": "COMPLETE",
  "result": {
    "content": "...",
    "latency_ms": 342,
    "agent_name": "worker-node-07"
  }
}
```

`agent_name` is load-bearing for multi-node routing and audit logging.

Appendix C: Verifier Ladder — Setup and Configuration

C.1 Rung R1: SymPy (zero additional setup)

SymPy ships with spl-llm:

```
pip install spl-llm          # SymPy included
spl3 run workflow.spl --kernel --llm ollama:gemma3
```

The `--kernel` flag starts an IPython kernel with `python3 kernelspec`. SymPy operations are registered as SPL tools in `cookbook/67_symbolic_math/tools.py` and are available to any workflow that imports that file or registers equivalent tools.

C.2 Rung R2: SageMath

SageMath requires a separate install due to its size (~1 GB):

```
# Option A --- conda (recommended)
conda install -c conda-forge sagemath

# Option B --- system package (Ubuntu/Debian)
sudo apt install sagemath

# Register the SageMath Jupyter kernelspec
sage -python -m sage.repl.ipynb_kernel.install

# Run with SageMath kernel
spl3 run workflow.spl --kernel --kernel-name sagemath --llm ollama:gemma3
```

SPL discovers the kernel via `ensure_kernelspec("sagemath")` in `spl3/kernel.py`. The same `SOLVE/ASSERT` primitives work unchanged; only the execution environment differs.

C.3 Rung R3: Lean 4 with Lean REPL

Lean 4 and the REPL are installed via the one-shot setup script:

```
# Full install: elan + Lean 4 + leanprover-community/repl (pinned v4.30.0)
bash cookbook/tools/lean/setup_lean.sh

# Optionally with Mathlib (~2 GB download, required for library search)
bash cookbook/tools/lean/setup_lean.sh --with-mathlib
```

The script installs `elan` (Lean version manager) to `$ELAN_HOME` (default `/opt/lean`) and builds the REPL with `lake`. The SPL Lean bridge (`spl3/lean_bridge.py`) wraps the REPL in a persistent session:

```
from spl3.lean_bridge import LeanREPL

repl = LeanREPL().start()          # stdlib only
repl = LeanREPL.mathlib().start()  # with Mathlib imports
```

The bridge passes 15/15 unit tests in `tests/test_lean_bridge.py`, covering statement checking, proof verification, `exact?` citation, timeout recovery, and REPL restart.

```
# Verify installation
pytest tests/test_lean_bridge.py -v
```



```
# Run recipe 76 (the full proof pipeline)
spl3 run cookbook/76_lean_proof/lean_proof.spl --kernel --llm claude_cli
```

C.4 Environment Variables

Variable	Default	Purpose
MOMAGRID_HUB_URL	http://localhost:9000	Momagrid hub endpoint
OPENROUTER_API_KEY	—	OpenRouter authentication
ELAN_HOME	/opt/lean	Lean toolchain root
SPL_LEAN_REPL_PATH	auto-detected	Path to <code>lean_repl</code> binary
SPL_KERNEL_TIMEOUT	120	Kernel execution timeout (seconds)
SPL_MAX_ITERATIONS	5	Default WHILE loop guard

Appendix D: Cookbook Recipe Catalog

The cookbook ships 65+ recipes across 12 categories. Active recipes (`is_active: true`) run in the batch runner (`python cookbook/run_all.py`).

D.1 Foundations (00–09)

ID	Name	Key construct
00	Hello World	<code>GENERATE</code>
01	Self-Refine	<code>WHILE</code> quality gate
02	Chain-of-Thought	<code>GENERATE</code> chain
03	ReAct Agent	<code>CALL</code> tool loop
04	Multi-Step Reasoning	<code>WORKFLOW</code> composition
05	Self-Refine (v2)	<code>EVALUATE</code> + <code>WHILE</code>
06	Debate / Multi-Agent	<code>CALL</code> <code>PARALLEL</code>
07	Tool-Use	<code>CREATE</code> <code>TOOL_API</code>
08	Exception Handling	<code>EXCEPTION</code> taxonomy
09	Memory + RAG	<code>memory()</code> , <code>rag()</code>

D.2 Content Generation (10–19)

ID	Name	Key construct
10	Blog Writer	<code>WORKFLOW</code> + <code>WHILE</code>
11	Code Generator	<code>GENERATE</code> + <code>ASSERT</code>
12	Summarizer	<code>SELECT</code> + budget
13	Translator	<code>GENERATE</code> multimodal
14	Report Builder	<code>CALL</code> composition
15	Story Generator	<code>EVALUATE</code> branching
16	Email Drafter	<code>WORKFLOW</code> + <code>COMMIT</code>
17	Document Q&A	<code>rag()</code> + <code>SELECT</code>
18	Meeting Notes	<code>GENERATE</code> extraction
19	Slide Outline	<code>CALL</code> chain

D.3 Analysis and Extraction (20–29)

ID	Name	Key construct
20	Data Extraction	<code>GENERATE</code> structured output
21	Sentiment Pipeline	<code>EVALUATE</code> classification
22	Text2SPL Demo	<code>text2spl</code> toolchain
23	Structured Output	type-annotated <code>GENERATE</code>
24	Few-Shot Prompting	<code>SELECT</code> with examples
25	Nested Procedures	<code>PROCEDURE</code> + <code>CALL</code>
26	Prompt A/B Test	<code>CALL</code> <code>PARALLEL</code>
27	Data Extraction (v2)	<code>SOLVE</code> post-processing
28	Customer Support Triage	<code>EVALUATE</code> routing
29	Meeting Notes to Actions	<code>GENERATE</code> + <code>COMMIT</code>

D.4 Education and Tutoring (30–39)

ID	Name	Key construct
30	Code Generator + Tests	GENERATE + ASSERT
31	Sentiment Pipeline (v2)	EVALUATE + WHILE
32	Socratic Tutor	WHILE dialogue loop
33	Interview Simulator	CALL role alternation
34	Progressive Summarizer	WHILE + length gate
35	Hypothesis Tester	SOLVE + ASSERT
36	Tool-Use / Function-Call	CREATE TOOL_API
37	Headline News Aggregator	CALL PARALLEL
38	Bedrock Quickstart	--adapter bedrock
39	Vertex AI Quickstart	--adapter vertex

D.5 Cloud and Infrastructure (40–49)

ID	Name	Key construct
40	Azure OpenAI Quickstart	--adapter azure
41	Human Steering	EXCEPTION + pause
42	Knowledge Synthesis	rag() + WHILE
43	Prompt Self-Tuning	EVALUATE + COMMIT
44	Adaptive Failover	EXCEPTION escalation
45	Vision to Action	generate_multimodal()
46	(reserved)	—
47	arXiv Morning Brief	CALL PARALLEL fetch
48	Credit Risk Assessment	SOLVE + ASSERT
49	Regulatory News Audit	EVALUATE compliance

D.6 Multimodal (50–59)

ID	Name	Key construct
50	Code Pipeline	GENERATE + ASSERT
51	(reserved)	—
52	(reserved)	—
53	Video Summary	generate_multimodal() VIDEO
54	Text to Image	generate_multimodal() IMAGE
55	Text to Speech	generate_multimodal() AUDIO
56	Text to Video	generate_multimodal() VIDEO
57	Image Format Conversion	CALL + SOLVE
58	Image Restyle	generate_multimodal()
59	Audio Format Conversion	CALL + SOLVE

D.7 Parallel and Compilation (60–69)

ID	Name	Key construct
60	(reserved)	—
61	Video to Audio	CALL + kernel

ID	Name	Key construct
62	Video to Image	<code>generate_multimodal()</code>
63	Parallel Code Review	<code>CALL PARALLEL</code>
64	Parallel News Digest	<code>CALL PARALLEL</code>
65	vibe-splc	<code>spl3 vibe + spl3 splc</code>
66	<i>(reserved)</i>	—
67	Symbolic Math Solver	<code>SOLVE + ASSERT</code> (SymPy)
68	<i>(reserved)</i>	—
69	<i>(reserved)</i>	—

D.8 Verifier Ladder and Concept-Book (70–76)

ID	Name	Key construct
70	Linalg Content Cache	<code>cache_get/cache_put</code> shared cache
71	Linalg Concept-Book	<code>splc --lang python/linalg</code>
72	Verify arXiv References	<code>SOLVE + ASSERT</code> citation check
73	Intro Geometry Concept-Book	<code>splc --lang python/intro_geometry</code>
74	Generic Concept-Book	domain YAML + <code>splc</code>
75	SageMath Solver	<code>SOLVE</code> (SageMath kernel)
76	Lean Proof Verifier	<code>SOLVE/ASSERT</code> (Lean 4, R3)

Appendix E: Experiment — Full Results and Methodology

E.1 Experimental Protocol

Two sessions were run on the same local workstation (Ollama for 9 local models + `claude_cli` for sonnet-4-6):

Session	Cells	Repeats	ID
Pilot	400	r=1	<code>exp-20260615-073849</code>
Repeated	1200	r=3	<code>exp-20260615-191224</code>

Pass oracle: SPL status codes — solver arm: `complete` (chain kernel-verified); LLM-only arm: `complete` or `unverified_success` (non-empty response)

Database: `cookbook/77_neurosymbolic/experiment_results.db` (SQLite)

The dual-arm design uses a single `.spl` workflow (`symbolic_math.spl`) with `enable_solver` as a runtime parameter. The harness (`run_experiment.py`) iterates all (model $\times$ problem $\times$ solver_mode) cells sequentially, invoking `spl3 run` for each and persisting structured results to the DB. Status codes reported in the DB:

- `complete` — full chain verified by the symbolic kernel (solver arm) or non-empty LLM response (LLM-only arm)
- `unverified_success` — LLM-only arm produced output but with `steps=0`
- `solver_error` — kernel rejected one or more steps (expression evaluation failed)
- `plan_error` — plan parsed but semantically invalid (e.g., wrong step count)

E.2 Problem Set

The 20 problems span six tiers across two backends. T0–T2 use SymPy (10 problems); T3–T5 use SageMath (10 problems). Each problem has a known ground-truth answer available in the kernel but not used as the pass oracle (status codes provide it).

Tier	Backend	Count	Category	Example problem
T0	SymPy	2	Single-step polynomial	$d/dx (x^4 - 2x^2 + 1)$; simplify $(x^2-1)/(x-1)$
T1	SymPy	4	Multi-step polynomial chains	expand $(x+1)^2 \rightarrow \text{factor}$; diff $3x^3 - x \rightarrow \text{factor} \rightarrow \text{solve}$
T2	SymPy	4	Transcendental / limits / series / trig	$\sin(x)/x$ as $x \rightarrow 0$; Taylor $\sin(x)$ deg 5
T3	Sage	4	Integration / linear systems / eigenvalues	$\sqrt{4-x^2} dx$; $x+y=5, x-y=1$; eig $[[1,2],[3,4]]$
T4	Sage	4	Laplace / ODEs / summation / roots	$L\{e^{-2t}\}$; $y=y, y(0)=1; \sum 1/n^2$; roots x^4-1
T5	Sage	2	Expert: 2nd-order ODE + transform verify	$y-3y'+2y''=0$; $L^1\{s/(s^2+4)\}$ then verify

E.3 Per-Tier Pass Rates (r=3 session, exp-20260615-191224)

Solver arm pass rate by model and tier (%) — mean over 3 runs/cell:

Model	T0	T1	T2	T3	T4	T5	Overall
gemma4:e2b	100	100	100	100	75	83	93
sonnet-4-6	100	92	100	100	58	50	85
rnj-1	83	100	100	50	92	50	82
gemma3	83	100	75	75	50	50	73
qwen2.5	100	100	92	67	33	33	72
phi4	67	75	100	50	50	50	67
llama3.2	67	83	100	42	42	50	65
deepseek-v2:16b	50	75	50	58	42	33	53
lfm2.5	67	33	25	50	25	33	37
phi3	33	33	33	42	33	0	32
Tier avg	75	79	77	63	50	43	—

LLM-only arm pass rate by model and tier (%) — mean over 3 runs/cell:

Model	T0	T1	T2	T3	T4	T5	Overall
sonnet-4-6	100	100	100	100	100	100	100
rnj-1	100	100	100	100	100	100	100
qwen2.5	100	100	100	100	100	100	100
phi4	100	100	100	100	100	100	100
phi3	100	100	100	100	100	100	100
gemma3	100	100	100	100	100	100	100
llama3.2	100	100	100	100	100	100	100
deepseek-v2:16b	100	100	100	100	100	100	100
gemma4:e2b	100	100	100	92	92	100	97
lfm2.5	100	67	75	92	75	50	77

E.4 Failure Mode Breakdown (Solver Arm, r=3, 60 runs per model)

Model	complete	solver_error	plan_error	runs
gemma4:e2b	56	0	4	60
sonnet-4-6	51	8	0	60
rnj-1	49	11	0	60
gemma3	44	16	0	60
qwen2.5	43	17	0	60
phi4	40	20	0	60
llama3.2	39	21	0	60
deepseek-v2:16b	32	28	0	60
lfm2.5	22	11	26	60
phi3	19	41	0	60

`plan_format_error` is zero across all models and both sessions. `lfm2.5` is the sole `plan_error` outlier (26/60 solver failures), indicating a plan-generation failure distinct from kernel expression errors. `phi3` accumulates 41 `solver_error` failures across 60 runs, the most of any model, reflecting its difficulty naming correct Sage operations.

E.5 r=1 vs r=3 Comparison (Pilot vs Repeated Run)

The pilot session (`exp-20260615-073849`, r=1) and repeated session (`exp-20260615-191224`, r=3) ran the identical 10-model $\times$ 20-problem $\times$ 2-arm design. Solver arm pass rates (overall %):

Model	r=1 (pilot)	r=3 (repeated)	Δ
gemma4:e2b	95	93	-2
sonnet-4-6	85	85	0
rnj-1	90	82	-8
gemma3	70	73	+3
qwen2.5	75	72	-3
phi4	85	67	-18
llama3.2	65	65	0
deepseek-v2:16b	45	53	+8
lfn2.5	45	37	-8
phi3	30	32	+2

Top-3 and bottom-2 rankings are stable. The largest swing is phi4 (-18 pp), revealing pilot over-estimation. `sonnet-4-6` and `llama3.2` are perfectly stable at 85% and 65% respectively, confirming deterministic solver behavior for those models. The r=3 session is the authoritative result used in §6.

E.6 Judge Prompt

The LLM sanity gate uses the following prompt for each run:

You are a mathematics judge. You will be given a problem and a proposed answer.
Determine whether the answer is mathematically correct.

Problem: {problem}

Proposed answer: {answer}

Reply with exactly one word: "pass" if the answer is correct, "fail" if it is not.
Do not explain. Do not add punctuation.

The judge model (`claude-sonnet-4-6`) is run independently of the experiment sessions and has no access to the SymPy ground truth. `qwen3` results are excluded from all tables: root-cause analysis confirmed that `qwen3.5:9b` runs in extended thinking mode by default, exhausting its token budget on internal deliberation before emitting any structured output. This is a model-interface incompatibility, not a capability failure — the thinking trace demonstrates correct reasoning. The re-run will use a non-thinking `qwen3` variant or disable thinking mode via the `/no_think` flag.

Appendix F: Compilation Pipeline — Target Examples

The `spl3 splc` compiler translates a `.spl` workflow to idiomatic code in each target framework. All examples below compile from the same source: a minimal two-mode workflow that decomposes a math problem and verifies it with SymPy.

Source: `verify_step.spl`

```
WORKFLOW verify_step
  INPUT: @problem TEXT
  OUTPUT: @report TEXT
DO
  GENERATE decompose(@problem) INTO @steps
    SELECT @problem AS problem
    PROMPT 'Decompose into expr|op steps, one per line: @@problem@@'

  SOLVE @result SYMPY := "solve_chain(@@steps@@)"

  ASSERT verify(@@result@@, @@problem@@)
    OTHERWISE RETURN @report WITH status = 'verification_failed'

  GENERATE narrate(@result, @problem) INTO @report
    SELECT @result AS result, @problem AS problem
    PROMPT 'Explain this verified result in plain language: @@result@@'

  RETURN @report WITH status = 'complete'
END
```

F.1 LangGraph Target

```
spl3 splc verify_step.spl --target langgraph

# verify_step_langgraph.py --- generated by spl3 splc
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    problem: str
    steps: str
    result: str
    report: str
    status: str

def node_decompose(state: State) -> State:
    prompt = f"Decompose into expr|op steps, one per line: {state['problem']}"
    state["steps"] = llm_call(prompt)
    return state

def node_solve(state: State) -> State:
    state["result"] = kernel_exec(f"solve_chain({state['steps']!r})")
    return state

def node_assert(state: State) -> State:
    ok = kernel_exec(f"bool(verify({state['result']!r}, {state['problem']!r}))")
```



```

    state["status"] = "ok" if ok == "True" else "verification_failed"
    return state

def node_narrate(state: State) -> State:
    prompt = f"Explain this verified result in plain language: {state['result']}"
    state["report"] = llm_call(prompt)
    return state

def route_assert(state: State) -> str:
    return "narrate" if state["status"] == "ok" else END

graph = StateGraph(State)
graph.add_node("decompose", node_decompose)
graph.add_node("solve", node_solve)
graph.add_node("assert_gate", node_assert)
graph.add_node("narrate", node_narrate)
graph.set_entry_point("decompose")
graph.add_edge("decompose", "solve")
graph.add_edge("solve", "assert_gate")
graph.add_conditional_edges("assert_gate", route_assert)
graph.add_edge("narrate", END)
app = graph.compile()

```

F.2 Go Target

```

spl3 splc verify_step.spl --target go

// verify_step.go --- generated by spl3 splc
package main

import (
    "fmt"
    "splruntime" // SPL Go runtime shim
)

type VerifyStepState struct {
    Problem string
    Steps   string
    Result  string
    Report  string
    Status  string
}

func VerifyStep(problem string) (string, error) {
    s := &VerifyStepState{Problem: problem}

    var err error
    s.Steps, err = splruntime.Generate("Decompose into expr|op steps, one per line: "+s.Problem)
    if err != nil { return "", err }

    s.Result, err = splruntime.KernelExec(fmt.Sprintf("solve_chain(%q)", s.Steps))
    if err != nil { return "", err }

    ok, err := splruntime.KernelAssert(

```

```

        fmt.Sprintf("bool(verify(%q, %q))", s.Result, s.Problem))
    if err != nil || !ok {
        return "", splruntime.CommitStatus("verification_failed")
    }

    s.Report, err = splruntime.Generate(
        "Explain this verified result in plain language: " + s.Result)
    if err != nil { return "", err }

    return s.Report, nil
}

```

F.3 TypeScript Target

```

spl3 splc verify_step.spl --target typescript

// verifyStep.ts --- generated by spl3 splc
import { generate, kernelExec, kernelAssert } from "./spl-runtime";

interface VerifyStepState {
    problem: string;
    steps?: string;
    result?: string;
    report?: string;
}

export async function verifyStep(problem: string): Promise<string> {
    const s: VerifyStepState = { problem };

    s.steps = await generate(
        'Decompose into expr|op steps, one per line: ${s.problem}'
    );

    s.result = await kernelExec('solve_chain(${JSON.stringify(s.steps)})');

    const ok = await kernelAssert(
        'bool(verify(${JSON.stringify(s.result)}), ${JSON.stringify(s.problem)})'
    );
    if (!ok) throw new Error("verification_failed");

    s.report = await generate(
        'Explain this verified result in plain language: ${s.result}'
    );
    return s.report;
}

```

F.4 DODA Invariant Across Targets

The .spl source is unchanged across all three targets. The compiler manages:

SPL construct	LangGraph	Go	TypeScript
GENERATE	StateGraph node	splruntime.Generate()	await generate()
SOLVE	StateGraph node	splruntime.KernelExec()	await kernelExec()

SPL construct	LangGraph	Go	TypeScript
ASSERT ... OTHERWISE	conditional edge	if !ok branch	if (!ok) throw
CALL PARALLEL	parallel nodes	goroutines + sync.WaitGroup	Promise.all([...])
RETURN (COMMIT deprecated)	END node	return / error	return / throw

The DODA principle holds at the compilation level: the same declarative specification produces idiomatic, correct code in each target language without any manual adaptation.

Appendix G: Runtime Implementation Details

G.1 Kernel Substrates

`spl3/kernel.py` provides two execution substrates for deterministic nodes:

IPythonKernel runs an out-of-process Jupyter kernel via `jupyter_client.KernelManager`. It is lazy-started on the first `SOLVE` or `ASSERT` encountered during a workflow run, then held alive for the run's duration. All state — imported modules, defined variables, intermediate results — persists across steps within a session. The kernel exposes a thread-locked `execute()` method that captures stdout and the `text/plain` repr of the last expression. Python-level errors raise `KernelExecutionError`, which the executor maps to the SPL exception hierarchy.

KernelSession is a lightweight in-process `exec()` substrate with a persistent namespace dict. It is used for `CREATE TOOL_API` body execution, where the overhead of an out-of-process kernel is unnecessary. Two isolation scopes are supported: `"session"` (shared namespace across the run) and `"workflow"` (fresh namespace per workflow invocation).

The kernel rung (SymPy / SageMath / Lean 4) is selected via `--kernel-name` at invocation time; setup instructions for each rung are in Appendix C.

G.2 Template Resolution and Kernel Dispatch Harness

Before a `SOLVE` expression is sent to the kernel, the executor resolves `@@var@@` substitutions:

```
def _resolve_python_template(self, expr: str, state: WorkflowState) -> str:
    return re.sub(
        r'@@(\w+)@@',
        lambda m: str(state.get_var(m.group(1))),
        expr
    )
```

The resolved expression is then wrapped in a standard harness before dispatch:

- **SOLVE:** `_spl_solve_result = {expr}; print(str(_spl_solve_result))` — printed output is captured and assigned to the target `@var`.
- **ASSERT:** `_spl_assert_result = bool({expr}); print(_spl_assert_result)` — the executor checks the output equals `"True"`; on mismatch the `OTHERWISE` body executes.

This wrapping is invisible to the workflow author: `SOLVE` and `ASSERT` behave as if Python expressions natively return SPL variables.

G.3 Adapter Bootstrap and Momagrid Protocol

The adapter bootstrap order in `spl3/adapters/__init__.py` is three-step:

1. **dd-llm bridge** — if installed, registered first for all cloud model prefixes (Anthropic, OpenAI, Google, Mistral, Cohere, Together, Groq, Fireworks, Perplexity, DeepSeek).
2. **Bespoke fallbacks** — `claude_cli` and `openrouter` registered as standalone adapters.
3. **Always-available** — `ollama` and `momagrid` registered unconditionally.

The Momagrid adapter submits tasks via `POST /tasks` and polls `GET /tasks/{id}`. The hub response carries a nested result object:

```
{"state": "COMPLETE", "result": {"content": "...", "latency_ms": 342, "agent_name": "worker-07"}}
```

`agent_name` is load-bearing for multi-node routing and audit logging; it must not be flattened into the top-level response.

G.4 Ecosystem Tooling

spl3 text2spl converts a natural-language description to valid **.spl** source using Code-RAG: the 65+ cookbook recipes are indexed as description/source pairs and retrieved by similarity. A validation loop parses the generated output, feeds parse errors back to the LLM, and iterates until syntactically valid.

spl3 vibe performs one-shot NL-to-working-code generation: natural language $\rightarrow$ **.spl** workflow $\rightarrow$ runnable Python implementation + README + test data. The `--out-dir` flag writes all artifacts to a folder; `--adapter` selects the generating model.

Both tools are available after `pip install spl-llm` with no additional configuration.

G.5 Lean 4 Three-Stage Protocol (SPL Source)

The full SPL implementation of the Lean rung referenced in §4:

```
-- Initialise the Lean REPL (once per session)
CALL run_python("from spl3.lean_bridge import LeanREPL;
                 _spl_lean = LeanREPL.mathlib().start();
                 print('ready')") INTO @lean_status

-- Stage 1: Formalize
GENERATE formalize_claim(@problem) INTO @lean_stmt
@check_code := f"print(_spl_lean.statement_ok(r'''{@lean_stmt}'''))"
CALL run_python(@check_code) INTO @stmt_ok

-- Stage 2: Typecheck with self-repair loop
@tries := 0
WHILE @tries < @max_tries DO
  EVALUATE @stmt_ok
  WHEN = "False" THEN
    GENERATE fix_formalization(@problem, @lean_stmt, @lean_feedback) INTO @lean_stmt
    CALL run_python(@check_code) INTO @stmt_ok
  ELSE
    @tries := @max_tries
  END
  @tries := @tries + 1
END

-- Stage 3: Prove with self-repair loop
GENERATE write_proof(@lean_stmt) INTO @lean_proof
@prove_code := f"print(_spl_lean.proof_ok(r'''{@lean_stmt}''', r'''{@lean_proof}'''))"
CALL run_python(@prove_code) INTO @proof_ok

@proof_tries := 0
WHILE @proof_tries < @max_tries DO
  EVALUATE @proof_ok
  WHEN = "False" THEN
    GENERATE fix_proof(@lean_stmt, @lean_proof, @lean_feedback) INTO @lean_proof
    CALL run_python(@prove_code) INTO @proof_ok
  ELSE
    @proof_tries := @max_tries
  END
  @proof_tries := @proof_tries + 1
END
```

```
RETURN @lean_proof WITH status = 'machine_proved'
```

Appendix I: Model Selection for Two-Mode Workflows

Two task types arise in a declarative two-mode workflow; they suit different model behaviors.

Property	Probabilistic	Deterministic
Goal	Pattern recognition, format mapping	Exact, reproducible derivation
Output	Approximate, context-shaped	Verifiable, same answer every run
Extended thinking	Liability — wastes tokens on work the kernel will do	N/A — correctness guaranteed by construction
Key metric	Structured output compliance	Machine-checkable correctness
Speed	Variable	Can be faster than probabilistic for structured problems
Role in SPL	LLM (GENERATE, EVALUATE)	Kernel (SOLVE, ASSERT)

In the SPL neurosymbolic experiment the LLM’s job is **probabilistic**: recognize the problem type and map it to **expr|op** format. The kernel’s job is **deterministic**: execute each step exactly. Thinking-mode models (qwen3.5:9b, deepseek-r1) apply deterministic-style deliberation to a probabilistic task, exhausting their token budget before emitting any structured output.

The broader implication: declarative composition lowers the LLM capability bar for math workflows. Without SPL, solving calculus requires a model that reasons deterministically. With SPL, it requires only a probabilistic translator — the kernel handles the derivation.

Appendix H: Working Plan

This section tracks open revision items. Remove before final submission.

Completed

- ☒ Paper structure: 8 main sections + Appendices A–G
- ☒ Title reframed: “Declarative Deterministic–Probabilistic Composition”
- ☒ Abstract, intro, contributions (C1–C4), conclusion aligned with title
- ☒ C5 (concept-book) and C6 (open-source) removed; focus on language and system
- ☒ Related Work: 6 subsections covering SQL precedent, imperative frameworks, declarative systems, neurosymbolic AI
- ☒ DODA corrected to “Design Once, Deploy Anywhere” (was “Declarative Once”)
- ☒ Implementation section condensed to 3 dual-mode-focused paragraphs; details moved to Appendix G
- ☒ Two-layer runtime subsection removed (reveals version evolution)
- ☒ References moved to immediately after Conclusion (before appendices)
- ☒ Citation numbers fixed: [4]/[5] placeholders → correct [6]–[16] in Related Work and Discussion
- ☒ [11] Auton placeholder removed; references renumbered [11]–[16]
- ☒ R2: `ensure_kernelspec()` code removed from main text → Appendix C reference
- ☒ R3: SPL protocol code condensed in main text → moved to Appendix G.5
- ☒ Appendix subsection starring fixed: all subsections after first `\section*{Appendix` are unnumbered
- ☒ `qwen3.5` renamed to `qwen3` for consistency with blog posts
- ☒ `qwen3` 0% failure root-caused: `qwen3.5:9b` thinking mode exhausts token budget before producing structured output; excluded from results
- ☒ `deepseek-r1` 40% failure root-caused: thinking mode cannot be suppressed even with explicit instructions; apparent 40% is partial-output artifact on simpler tiers; excluded from re-run. Both are classified as *thinking-mode token exhaustion* — a fourth failure mode distinct from format/planning/execution failures (documented in Appendix I)
- ☒ Appendices A–G drafted (EBNF grammar, adapter system, verifier ladder setup, cookbook catalog, experiment results, compilation pipeline, runtime details)
- ☒ **System 1/2 framing integrated** — added to §1 (Two-Mode Gap), §3 (P2, Two-Mode Composition Pattern), §7 (new “System 1/2 Decomposition” subsection). Clarified that System 1 = probabilistic/intuitive (not necessarily fast), System 2 = deterministic/mechanical (not necessarily slow). Added Kahneman [17].
- ☒ **Four-stage PLAN/SOLVE/ASSERT/EXPLAIN lifecycle** — made explicit in §3 Two-Mode Composition Pattern with per-stage annotations and explanatory prose.
- ☒ **Formalization boundary** — named and explained in §3 as the `@@var@@` seam where System 1 outputs are cast into System 2 inputs.
- ☒ **System 1/2 capability hierarchy** — discussed in §7, connecting F1 finding (format compliance failure) to the System 1/2 boundary.

In Progress

- ☒ **400-cell pilot completed** (exp-20260615-073849) — 10 models $\times$ 20 problems $\times$ 2 arms $\times$ r=1. Key results: gemma4:e2b 95%, rnj-1 90%, phi4 85%, sonnet-4-6 85%. Zero plan_format_error. SymPy 78% / Sage 57%.
- ☒ **1200-cell repeated run completed** (exp-20260615-191224) — same design $\times$ r=3. Key results: gemma4:e2b 93%, sonnet-4-6 85% (stable), rnj-1 82%, phi4 67% (−18pp from pilot). SymPy 78% / Sage 54%. Ranking top-3 and bottom-2 stable; phi4 pilot inflation confirmed.
- ☒ **§6 updated** with r=3 results as authoritative; F1–F5 revised; repeated-run stability discussion added to §6.4
- ☒ **Appendix E updated** — E.3/E.4 show r=3 data; E.5 adds r=1 vs r=3 comparison table; E.1 documents both sessions
- ☐ **Proof-read full paper** — check flow, clarity, and consistency section by section (in progress)

Pending

- ☐ **New Related Work — literature search follow-up** — read and integrate 5 Tier-1 papers identified in review-feedback/claude-research-20260614.md: Compiled AI (arXiv:2604.05150), Blueprint First (arXiv:2508.02721), DUPLEX (arXiv:2603.23909), MCP-Solver (arXiv:2501.00539), LLM+P (arXiv:2304.11477). Add ~3 new paragraphs to Related Work (“Declarative LLM Systems” extension + new “LLM+Solver Composition” subsection + one-liners in “Neurosymbolic AI”). Add ~7 new references. Also consider Tier-2: Compositional AI ASPLOS 2026, Symbolic Seams (arXiv:2603.15087), VERAfi (arXiv:2512.14744).
- ☐ **Figures** — consider adding: (1) execution pipeline diagram (two-mode dispatch); (2) verifier ladder schematic (R1→R2→R3); (3) pass-rate heatmap (models $\times$ tiers)
- ☐ **Fill reference placeholders** — verify full author lists, venues, and years for [9] Agent Spec, [10] eBay DSL, [11] Agentics 2.0
- ☐ **LLM-as-judge upgrade** — plan deterministic SymPy ground-truth comparison as follow-up to §6.3 limitation note
- ☐ **TMLR submission metadata** — fill \def\month{}, \def\year{}, \def\openreview{} in build_md2tex.sh after acceptance
- ☐ **Camera-ready switch** — change \usepackage{tmlr} $\rightarrow$ \usepackage[accepted]{tmlr} in preamble
- ☐ **Remove this appendix** before final submission

Venue Decision

See readme-venue.md for the ranked venue table, deadlines, positioning notes, and sources.